

Parallel and Distributed Computing

Assignment 1

Question 1: DAXPY Loop

Problem Statement:

DAXPY Loop: D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^{16} each, and P stands for Plus. The operation performed in one iteration is:

$$X[i] = A \times X[i] + Y[i]$$

The objective is to compare the speedup (in execution time) gained by increasing the number of threads starting from a 2-thread implementation.

```

1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int N = 65536;
6      double X[N], Y[N];
7      double a = 2.5;
8      int i, threads;
9      double t1, t2;
10     double seq_time;
11
12     for (i = 0; i < N; i++) {
13         X[i] = 1.0;
14         Y[i] = 2.0;
15     }
16
17     // measure sequential time
18     for (i = 0; i < N; i++)
19         X[i] = 1.0;
20
21     t1 = omp_get_wtime();
22     for (i = 0; i < N; i++) {
23         X[i] = a * X[i] + Y[i];
24     }
25     t2 = omp_get_wtime();
26     seq_time = t2 - t1;
27
28     printf("Sequential Time = %f seconds\n\n", seq_time);
29
30     // parallel execution with 2 to 12 threads
31     for (threads = 2; threads <= 12; threads++) {
32         double par_time, speedup;
33
34         omp_set_num_threads(threads);
35
36         for (i = 0; i < N; i++)
37             X[i] = 1.0;
38
39         t1 = omp_get_wtime();
40         #pragma omp parallel for
41         for (i = 0; i < N; i++) {
42             X[i] = a * X[i] + Y[i];
43         }
44         t2 = omp_get_wtime();
45
46         par_time = t2 - t1;
47         speedup = seq_time / par_time;
48
49         printf("Threads = %d   Time = %f   Speedup = %.2f\n", threads, par_time, speedu
50 p);}
51
52     return 0;
53 }
54

```

```

1 Sequential Time = 0.000532 seconds
2
3 Threads = 2    Time = 0.000292    Speedup = 1.82
4 Threads = 3    Time = 0.000183    Speedup = 2.91
5 Threads = 4    Time = 0.000188    Speedup = 2.83
6 Threads = 5    Time = 0.000335    Speedup = 1.59
7 Threads = 6    Time = 0.000614    Speedup = 0.87
8 Threads = 7    Time = 0.001655    Speedup = 0.32
9 Threads = 8    Time = 0.002700    Speedup = 0.20
10 Threads = 9   Time = 0.000321    Speedup = 1.66
11 Threads = 10  Time = 0.000449    Speedup = 1.18
12 Threads = 11  Time = 0.000445    Speedup = 1.19
13 Threads = 12  Time = 0.000526    Speedup = 1.01

```

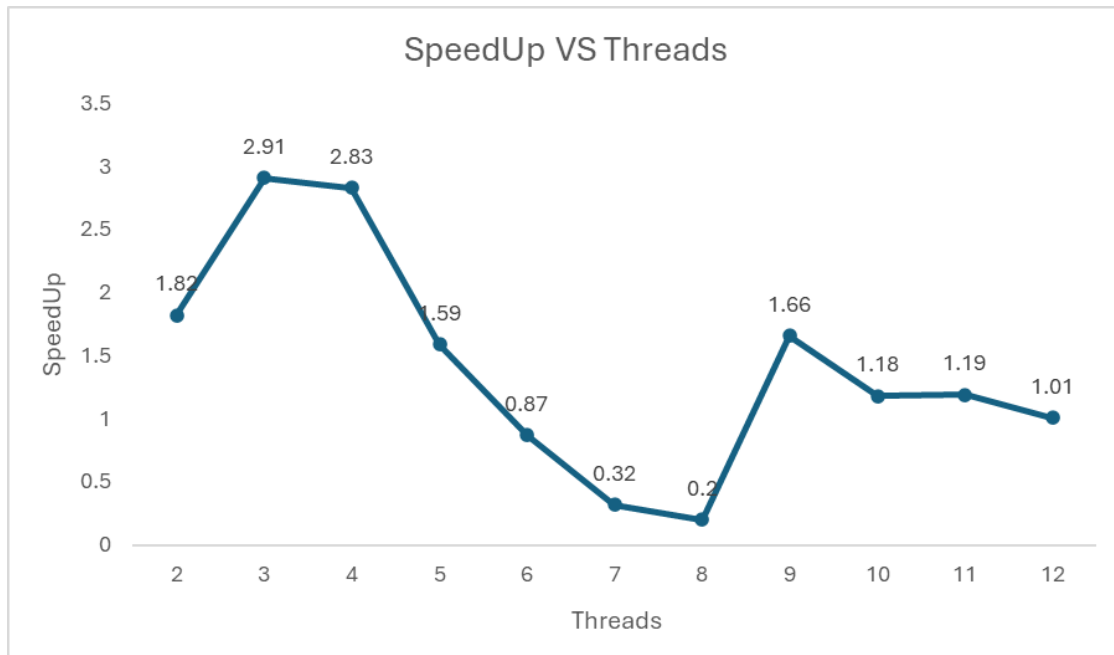
Observation

Speedup is still 4 threads and achieves maximum speedup at around 3–4 cores (approximately 2.9). The synchronization overhead is small. Since the laptop has 4 physical cores, execution remains within the physical core limit.

After 4 threads (5–8 threads), a stable dip in speedup is observed. This occurs because execution goes beyond physical cores and hyperthreading is used. Hyperthreading introduces thread creation and synchronization overhead, leading to memory bottlenecks and cache conflicts.

At around 9 threads, slight recovery is observed as the scheduler adapts, but speedup remains lower than the maximum achieved.

The DAXPY loop is memory bound and executes very fast, hence overhead dominates computation.



Conclusion

Best performance is achieved using 1–4 threads corresponding to physical cores. Beyond this point, parallelization overhead outweighs computation.

Question2:MatrixMultiplication

ProblemStatement:

Build a parallel implementation of multiplication of large matrices (e.g., 1000×1000). Repeat the experiment from Question 1 and analyze work partitioning among threads using:

- 1D threading
- 2D threading

```

1  #include <stdio.h>
2  #include <omp.h>
3
4  double A[1000][1000], B[1000][1000], C[1000][1000];
5
6  int main()
7  {
8      int N = 1000;
9      int i, j, k, threads;
10     double t1, t2, seq_time;
11
12     for (i = 0; i < N; i++)
13     {
14         for (j = 0; j < N; j++)
15         {
16             A[i][j] = 1.0;
17             B[i][j] = 2.0;
18             C[i][j] = 0.0;
19         }
20     }
21
22     // sequential
23     printf("Sequential:\n");
24     t1 = omp_get_wtime();
25     for (i = 0; i < N; i++)
26     {
27         for (j = 0; j < N; j++)
28         {
29             for (k = 0; k < N; k++)
30             {
31                 C[i][j] += A[i][k] * B[k][j];
32             }
33         }
34     }
35     t2 = omp_get_wtime();
36     seq_time = t2 - t1;
37     printf("Time = %f\n\n", seq_time);
38
39     // 1D parallel
40     printf("1D Parallel:\n");
41     for (threads = 2; threads <= 10; threads++)
42     {
43         double par_time, speedup;
44
45         omp_set_num_threads(threads);
46
47         for (i = 0; i < N; i++)
48             for (j = 0; j < N; j++)
49                 C[i][j] = 0.0;
50
51         t1 = omp_get_wtime();
52         #pragma omp parallel for private(j, k)
53         for (i = 0; i < N; i++)

```

```

1  {
2      for (j = 0; j < N; j++)
3      {
4          for (k = 0; k < N; k++)
5          {
6              C[i][j] += A[i][k] * B[k][j];
7          }
8      }
9  }
10     t2 = omp_get_wtime();
11
12     par_time = t2 - t1;
13     speedup = seq_time / par_time;
14
15     printf("Threads=%d Time=%f Speedup=%.2f\n", threads, par_time, speedup);
16 }
17
18 // 2D parallel
19 printf("\n2D Parallel:\n");
20 for (threads = 2; threads <= 10; threads++)
21 {
22     double par_time, speedup;
23
24     omp_set_num_threads(threads);
25
26     for (i = 0; i < N; i++)
27         for (j = 0; j < N; j++)
28             C[i][j] = 0.0;
29
30     t1 = omp_get_wtime();
31 #pragma omp parallel for collapse(2) private(k)
32     for (i = 0; i < N; i++)
33     {
34         for (j = 0; j < N; j++)
35         {
36             for (k = 0; k < N; k++)
37             {
38                 C[i][j] += A[i][k] * B[k][j];
39             }
40         }
41     }
42     t2 = omp_get_wtime();
43
44     par_time = t2 - t1;
45     speedup = seq_time / par_time;
46
47     printf("Threads=%d Time=%f Speedup=%.2f\n", threads, par_time, speedup);
48 }
49
50 return 0;
51 }
52

```



```
1 Sequential:
2 Time = 7.773309
3
4 1D Parallel:
5 Threads=2 Time=4.312744 Speedup=1.80
6 Threads=3 Time=3.151255 Speedup=2.47
7 Threads=4 Time=3.574226 Speedup=2.17
8 Threads=5 Time=2.714597 Speedup=2.86
9 Threads=6 Time=2.642308 Speedup=2.94
10 Threads=7 Time=2.780996 Speedup=2.80
11 Threads=8 Time=2.466476 Speedup=3.15
12 Threads=9 Time=2.566586 Speedup=3.03
13 Threads=10 Time=2.686164 Speedup=2.89
14
15 2D Parallel:
16 Threads=2 Time=4.730801 Speedup=1.64
17 Threads=3 Time=3.271672 Speedup=2.38
18 Threads=4 Time=3.160151 Speedup=2.46
19 Threads=5 Time=2.605118 Speedup=2.98
20 Threads=6 Time=2.594382 Speedup=3.00
21 Threads=7 Time=2.527724 Speedup=3.08
22 Threads=8 Time=2.466996 Speedup=3.15
23 Threads=9 Time=2.576557 Speedup=3.02
24 Threads=10 Time=2.643251 Speedup=2.94
```

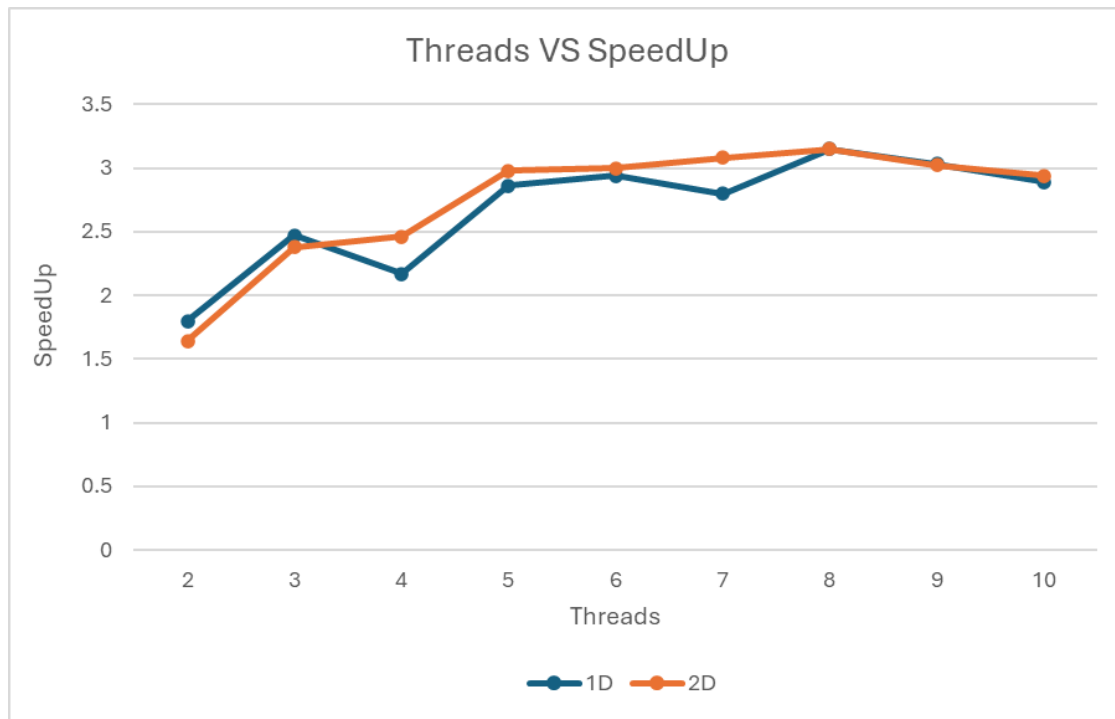
Observation

The sequential version uses a triple nested loop to compute each output element.

In the 1D parallel implementation, the outer loop is parallelized and each thread computes a subset of rows, resulting in low overhead.

In the 2D parallel implementation, both (i,j) indices are parallelized. This distributes work more evenly but increases scheduling overhead.

For 2–3 threads, overhead dominates and 1D performs better. For 4–8 threads, 2D slightly outperforms 1D due to improved load balancing. Cache effects and memory bandwidth limit perfect scaling.



Conclusion

Both 1D and 2D implementations are correct and scale well up to logical cores. Similar performance is observed due to moderate workload size.

Question 3: Calculation of π

Problem Statement:

The value of π is computed using numerical integration:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx$$

The integral is approximated using the rectangle method and parallelized using OpenMP reduction.

```

1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      long num_steps = 100000;
6      double step;
7      int i, threads;
8      double x, pi, sum;
9      double t1, t2, seq_time;
10
11     step = 1.0 / (double)num_steps;
12
13     // sequential
14     sum = 0.0;
15     t1 = omp_get_wtime();
16     for (i = 0; i < num_steps; i++) {
17         x = (i + 0.5) * step;
18         sum = sum + 4.0 / (1.0 + x * x);
19     }
20     pi = step * sum;
21     t2 = omp_get_wtime();
22     seq_time = t2 - t1;
23
24     printf("Sequential:\n");
25     printf("Pi = %.10f\n", pi);
26     printf("Time = %f\n\n", seq_time);
27
28     // parallel
29     printf("Parallel:\n");
30     for (threads = 2; threads <= 10; threads++) {
31         double par_time, speedup;
32
33         omp_set_num_threads(threads);
34
35         sum = 0.0;
36         t1 = omp_get_wtime();
37         #pragma omp parallel for private(x) reduction(+:sum)
38         for (i = 0; i < num_steps; i++) {
39             x = (i + 0.5) * step;
40             sum = sum + 4.0 / (1.0 + x * x);
41         }
42         pi = step * sum;
43         t2 = omp_get_wtime();
44
45         par_time = t2 - t1;
46         speedup = seq_time / par_time;
47
48         printf("Threads=%d Pi=%.10f Time=%f Speedup=%.2f\n", threads, pi, par_time, speedu
49 p);}
50
51     return 0;
52 }
53

```



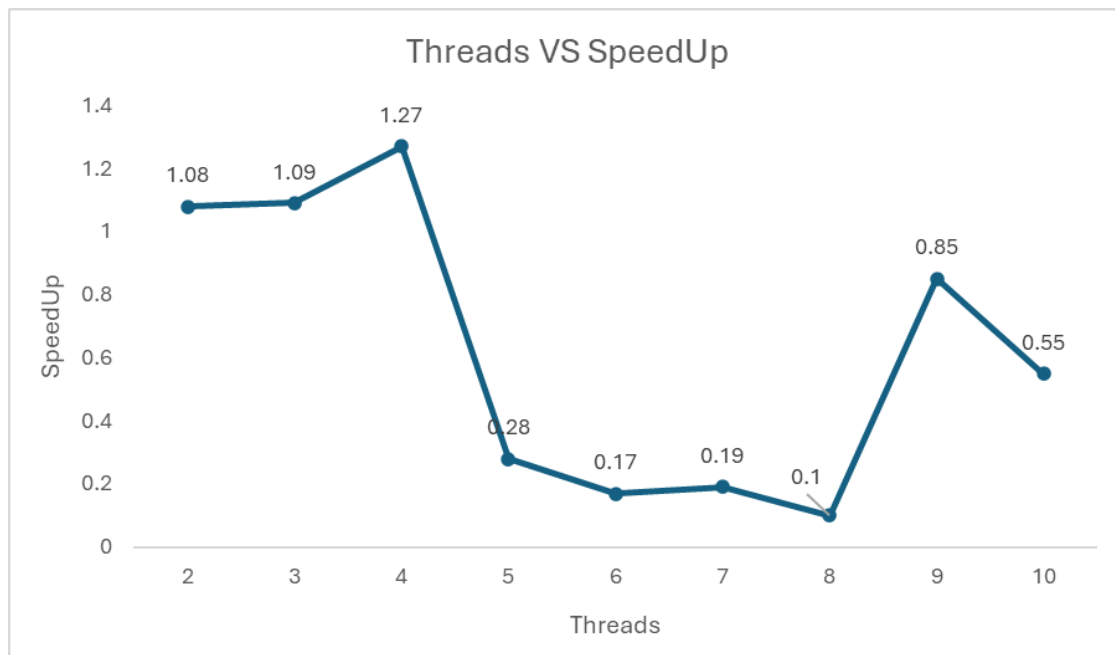
```
1 Sequential:
2 Pi = 3.1415926536
3 Time = 0.000232
4
5 Parallel:
6 Threads=2 Pi=3.1415926536 Time=0.000215 Speedup=1.08
7 Threads=3 Pi=3.1415926536 Time=0.000212 Speedup=1.09
8 Threads=4 Pi=3.1415926536 Time=0.000183 Speedup=1.27
9 Threads=5 Pi=3.1415926536 Time=0.000829 Speedup=0.28
10 Threads=6 Pi=3.1415926536 Time=0.001326 Speedup=0.17
11 Threads=7 Pi=3.1415926536 Time=0.001207 Speedup=0.19
12 Threads=8 Pi=3.1415926536 Time=0.002323 Speedup=0.10
13 Threads=9 Pi=3.1415926536 Time=0.000274 Speedup=0.85
14 Threads=10 Pi=3.1415926536 Time=0.000419 Speedup=0.55
```

Observation

The loop iterations are divided among threads and partial sums are recombined using `reduction(+:sum)`, avoiding race conditions.

Maximum speedup is small (~ 1.27) because the sequential execution time is extremely small. For 2–4 threads, speedup increases gradually.

For 5–8 threads, speedup drops due to synchronization overhead, cache contention, and reduction costs. A slight recovery at higher thread counts is attributed to scheduling effects.



Conclusion

The program computes an accurate approximation of π . Parallelism is beneficial only up to physical cores, after which overhead dominates.

Submitted to :Dr Saif Nalband

Submitted by :Gautam Tayal

Roll No.:102483024